

第2章 线性表

一、选择题

1. 下述哪一条是顺序存储结构的优点? ()
A. 存储密度大 B. 插入运算方便 C. 删除运算方便
D. 可方便地用于各种逻辑结构的存储表示
2. 下面关于线性表的叙述中, 错误的是哪一个? ()
A. 线性表采用顺序存储, 必须占用一片连续的存储单元。
B. 线性表采用顺序存储, 便于进行插入和删除操作。
C. 线性表采用链接存储, 不必占用一片连续的存储单元。
D. 线性表采用链接存储, 便于插入和删除操作。
3. 线性表是具有 n 个 () 的有限序列 ($n>0$)。
A. 表元素 B. 字符 C. 数据元素 D. 数据项 E. 信息项
4. 若某线性表最常用的操作是存取任一指定序号的元素和在最后进行插入和删除运算, 则利用 () 存储方式最节省时间。
A. 顺序表 B. 双链表
C. 带头结点的双循环链表 D. 单循环链表
5. 某线性表中最常用的操作是在最后一个元素之后插入一个元素和删除第一个元素, 则采用 () 存储方式最节省运算时间。
A. 单链表 B. 仅有头指针的单循环链表
C. 双链表 D. 仅有尾指针的单循环链表
6. 链表不具有的特点是 ()
A. 插入、删除不需要移动元素 B. 可随机访问任一元素
C. 不必事先估计存储空间 D. 所需空间与线性长度成正比
7. 若长度为 n 的线性表采用顺序存储结构, 在其第 i 个位置插入一个新元素的算法的时间复杂度为 () ($1 \leq i \leq n+1$)。
A. $O(0)$ B. $O(1)$ C. $O(n)$ D. $O(n^2)$
8. 对于顺序存储的线性表, 访问结点和增加、删除结点的时间复杂度为 ()。
A. $O(n)$ $O(n)$ B. $O(n)$ $O(1)$ C. $O(1)$ $O(n)$ D. $O(1)$ $O(1)$
9. 线性表 ($a_1, a_2, \dots, a_n$) 以链接方式存储时, 访问第 i 位置元素的时间复杂性为 ()
A. $O(i)$ B. $O(1)$ C. $O(n)$ D. $O(i-1)$
10. 在一个以 h 为头的单循环链中, p 指针指向链尾的条件是 ()
A. $p \rightarrow next = h$ B. $p \rightarrow next = NIL$ C. $p \rightarrow next \rightarrow next = h$ D. $p \rightarrow data = -1$
11. 完成在双循环链表结点 p 之后插入 s 的操作是 ();
A. $p \rightarrow next = s$; $s \rightarrow priou = p$; $p \rightarrow next \rightarrow priou = s$; $s \rightarrow next = p \rightarrow next$;
B. $p \rightarrow next \rightarrow priou = s$; $p \rightarrow next = s$; $s \rightarrow priou = p$; $s \rightarrow next = p \rightarrow next$;
C. $s \rightarrow priou = p$; $s \rightarrow next = p \rightarrow next$; $p \rightarrow next = s$; $p \rightarrow next \rightarrow priou = s$;
D. $s \rightarrow priou = p$; $s \rightarrow next = p \rightarrow next$; $p \rightarrow next \rightarrow priou = s$; $p \rightarrow next = s$;
12. 在单链表指针为 p 的结点之后插入指针为 s 的结点, 正确的操作是: ()。
A. $p \rightarrow next = s$; $s \rightarrow next = p \rightarrow next$; B. $s \rightarrow next = p \rightarrow next$; $p \rightarrow next = s$;
C. $p \rightarrow next = s$; $p \rightarrow next = s \rightarrow next$; D. $p \rightarrow next = s \rightarrow next$; $p \rightarrow next = s$;
13. 对于一个头指针为 $head$ 的带头结点的单链表, 判定该表为空表的条件是 ()
A. $head == NULL$ B. $head \rightarrow next == NULL$ C. $head \rightarrow next == head$ D. $head != NULL$
14. 在双向链表存储结构中, 删除 p 所指的结点时须修改指针 ()。
A. $(p \rightarrow llink) \rightarrow rlink = p \rightarrow rlink$ $(p \rightarrow rlink) \rightarrow llink = p \rightarrow llink$;

- B. $p \rightarrow llink = (p \rightarrow llink) \rightarrow llink$ $(p \rightarrow llink) \rightarrow rlink = p$;
 C. $(p \rightarrow rlink) \rightarrow llink = p$ $p \rightarrow rlink = (p \rightarrow rlink) \rightarrow rlink$
 D. $p \rightarrow rlink = (p \rightarrow llink) \rightarrow llink$ $p \rightarrow llink = (p \rightarrow rlink) \rightarrow rlink$;

二、判断

1. 链表中的头结点仅起到标识的作用。()
2. 顺序存储结构的主要缺点是不利于插入或删除操作。()
3. 线性表采用链表存储时, 结点和结点内部的存储空间可以是不连续的。()
4. 顺序存储方式插入和删除时效率太低, 因此它不如链式存储方式好。()
5. 顺序存储方式只能用于存储线性结构。()
6. 所谓静态链表就是一直不发生变化的链表。()
7. 顺序存储方式的优点是存储密度大, 且插入、删除运算效率高。()
8. 链表是采用链式存储结构的线性表, 进行插入、删除操作时, 在链表中比在顺序存储结构中效率高。()
9. 线性表的特点是每个元素都有一个前驱和一个后继。()
10. 取线性表的第 i 个元素的时间同 i 的大小有关。()
11. 循环链表不是线性表。()
12. 线性表只能用顺序存储结构实现。()
13. 线性表就是顺序存储的表。()

三、填空

1. 当线性表的元素总数基本稳定, 且很少进行插入和删除操作, 但要求以最快的速度存取线性表中的元素时, 应采用_____存储结构。
2. 线性表 $L = (a_1, a_2, \dots, a_n)$ 用数组表示, 假定删除表中任一元素的概率相同, 则删除一个元素平均需要移动元素的个数是_____。
3. 设单链表的结点结构为 $(data, next)$, $next$ 为指针域, 已知指针 px 指向单链表中 $data$ 为 x 的结点, 指针 py 指向 $data$ 为 y 的新结点, 若将结点 y 插入结点 x 之后, 则需要执行以下语句: _____; _____;
4. 在一个长度为 n 的顺序表中第 i 个元素 ($1 \leq i \leq n$) 之前插入一个元素时, 需向后移动_____个元素
5. 在单链表中设置头结点的作用是_____。
6. 对于一个具有 n 个结点的单链表, 在已知的结点 $*p$ 后插入一个新结点的时间复杂度为_____, 在给定值为 x 的结点后插入一个新结点的时间复杂度为_____。
7. 根据线性表的链式存储结构中每一个结点包含的指针个数, 将线性链表分成_____和_____; 而又根据指针的连接方式, 链表又可分成_____和_____。
8. 在双向循环链表中, 向 p 所指的结点之后插入指针 f 所指的结点, 其操作是_____, _____, _____, _____。
9. 顺序存储结构是通过_____表示元素之间的关系的; 链式存储结构是通过_____表示元素之间的关系的。
10. 对于双向链表, 在两个结点之间插入一个新结点需修改的指针共 _____个, 单链表为_____个。
11. 循环单链表的最大优点是: _____。
12. 带头结点的双循环链表 L 中只有一个元素结点的条件是: _____
13. 在单链表 L 中, 指针 p 所指结点有后继结点的条件是: _____
14. 带头结点的双循环链表 L 为空表的条件是: _____。

四、应用题

1. 下面是一算法的核心部分, 试说明该算法的功能。

```
pre = L->next;
{L 是一单链表, 结点有数据域 data 和指针域 next}
IF pre <> Null THEN
  WHILE pre->next != Null DO
```

```

{p=pre->next;
  IF p->data>=pre->data THEN pre=p ELSE return(false)}
return(true);

```

2. 有线性表 $(a_1, a_2, \dots, a_n)$, 采用单链表存储, 头指针为 H, 每个结点中存放线性表中一个元素, 现查找某个元素值等于 X 的结点。分别写出下面三种情况的查找语句。要求时间尽量少。

(1) 线性表中元素无序。(2) 线性表中元素按递增有序。(3) 线性表中元素按递减有序。

3. 一线性表存储在带头结点的双向循环链表中, L 为头指针。如下算法:

(1) 说明该算法的功能。(2) 在空缺处填写相应的语句。

```

void unknown (BNODETP *L)
{
    ...
    p=L->next; q=p->next; r=q->next;
    while (q!=L)
    { while (p!=L) && (p->data>q->data) p=p->prior;
      q->prior->next=r; (1)_____;
      q->next=p->next; q->prior=p;
      (2)_____; (3)_____; q=r; p=q->prior;
      (4)_____;
    }
}

```

五、算法设计题

1. 假设有两个按元素值递增次序排列的线性表, 均以单链表形式存储。请编写算法将这两个单链表归并为一个按元素值递减次序排列的单链表, 并要求利用原来两个单链表的结点存放归并后的单链表。
2. 知 L1、L2 分别为两循环单链表的头结点指针, m,n 分别为 L1、L2 表中数据结点个数。要求设计一算法, 用最快的速度将两表合并成一个带头结点的循环单链表。
3. 设计算法将一个带头结点的单链表 A 分解为两个具有相同结构的链表 B、C, 其中 B 表的结点为 A 表中值小于零的结点, 而 C 表的结点为 A 表中值大于零的结点 (链表 A 的元素类型为整型, 要求 B、C 表利用 A 表的结点)。
4. 已知线性表 $(a_1 a_2 a_3 \dots a_n)$ 按顺序存于内存, 每个元素都是整数, 试设计用最少时间把所有值为负数的元素移到全部正数值元素前边的算法: 例: $(x, -x, -x, x, x, -x \dots x)$ 变为 $(-x, -x, -x \dots x, x, x)$ 。
5. 已知非空线性链表由 list 指出, 链结点的构造为 (data,link) .请写一算法, 将链表中数据域值最小的那个链结点移到链表的最前面。要求: 不得额外申请新的链结点。
6. 设线性表存于 A[1..size] 的前 num 各分量中, 且递增有序。请设计一个算法, 将 x 插入到线性表的适当位置上, 以保持线性表的有序性, 并在设计前说明设计思想, 最后说明所设计算法的时间复杂度。
7. 设有一个由正整数组成的无序 (向后) 单链表, 编写完成下列功能的算法:
 - (1) 找出最小值结点, 且打印该数值;
 - (2) 若该数值是奇数, 则将其与直接后继结点的数值交换;
 - (3) 若该数值是偶数, 则将其直接后继结点删除。
8. 已知三个带头结点的线性链表 A、B 和 C 中的结点均依元素值自小至大非递减排列 (可能存在两个以上值相同的结点), 编写算法对 A 表进行如下操作: 使操作后的链表 A 中仅留下三个表中均包含的数据元素的结点, 且没有值相同的结点, 并释放所有无用结点。限定算法的时间复杂度为 $O(m+n+p)$, 其中 m、n 和 p 分别为三个表的长度。

参考答案

一、选择题

1	A	2	B	3	C	4	A	5	D	6	B	7	C
8	C	9	C	10	A	11	D	12	B	13	B	14	A

二、判断题

1	×	2	√	3	√	4	×	5	×	6	×	7	×
8	√	9	×	10	×	11	×	12	×	13	×		

三、填空题

1. 顺序	2. $(n-1)/2$
3. $py \rightarrow next = px \rightarrow next; px \rightarrow next = py$	4. $n-i+1$
5. 主要是使插入和删除等操作统一, 在第一个元素之前插入元素和删除第一个结点不必另作判断。另外, 不论链表是否为空, 链表指针不变。	6. $O(1), O(n)$
7. 单链表, 多重链表, (动态) 链表, 静态链表	8. $f \rightarrow next = p \rightarrow next; f \rightarrow prior = p;$ $p \rightarrow next \rightarrow prior = f; p \rightarrow next = f;$
9. 物理上相邻 指针	10. 4 2
11. 从任一结点出发都可访问到链表中每一个元素。	12. $L \rightarrow next \rightarrow next == L$
13. $p \rightarrow next != null$	14. $L \rightarrow next == L \ \&\& \ L \rightarrow prior == L$

四、应用题

1. 该算法的功能是判断链表 L 是否是非递减有序, 若是则返回“true”; 否则返回“false”。pre 指向当前结点, p 指向 pre 的后继。

```
2. (1) while(p!=null && p->data!=X) p=p->next;
    if(p==null) return(null); // 查找失败
    else return(p); // 查找成功
    (2) while(p!=null && p->data<X) p=p->next;
    if(p==null || p->data>X) return(null); // 查找失败
    else return(p);
    (3) while(p!=null && p->data>X) p=p->next;
    if(p==null || p->data<X) return(null); // 查找失败
    else return(p); // 查找成功
```

3. 1) 本算法功能是将双向循环链表结点的数据域按值从小到大排序, 成为非递减 (可能包括数据域值相等的结点) 有序双向循环链表。

```
2) (1) r->prior=q->prior; // 将 q 结点摘下, 以便插入到适当位置。
    (2) p->next->prior=q; // (2) (3) 将 q 结点插入
    (3) p->next=q;
    (4) r=r->next; 或 r=q->next; // 后移指针, 再将新结点插入到适当位置。
```

五、算法设计题

1. [题目分析] 因为两链表已按元素值递增次序排列, 将其合并时, 均从第一个结点起进行比较, 将小的链入链表中, 同时后移链表工作指针。该问题要求结果链表按元素值递减次序排列。故在合并的同时, 将链表结点逆置。

LinkedList Union(LinkedList la, lb)

// la, lb 分别是带头结点的两个单链表的头指针, 链表中的元素值按递增序排列, 本算法将两链表合并成一个按元

素值递减次序排列的单链表。

```
{ pa=la->next; pb=lb->next; // pa, pb 分别是链表 la 和 lb 的工作指针
  la->next=null;           // la 作结果链表的头指针, 先将结果链表初始化为空。
  while(pa!=null && pb!=null) // 当两链表均不为空时作
  {
    if(pa->data<=pb->data)
    {
      r=pa->next;           // 将 pa 的后继结点暂存于 r。
      pa->next=la->next;     // 将 pa 结点链于结果表中, 同时逆置。
      la->next=pa;
      pa=r;                 // 恢复 pa 为当前待比较结点。
    }
    else
    {
      r=pb->next; // 将 pb 的后继结点暂存于 r。
      pb->next=la->next; // 将 pb 结点链于结果表中, 同时逆置。
      la->next=pb;
      pb=r; // 恢复 pb 为当前待比较结点。
    }
    while(pa!=null) // 将 la 表的剩余部分链入结果表, 并逆置。
    {
      r=pa->next; pa->next=la->next; la->next=pa; pa=r; }
    while(pb!=null)
    {
      r=pb->next; pb->next=la->next; la->next=pb; pb=r; }
  } // 算法 Union 结束。
```

[算法讨论] 上面两链表均不为空的表达式也可简写为 **while**(pa&&pb), 两递增有序表合并成递减有序表时, 上述算法是边合并边逆置。也可先合并完, 再作链表逆置。后者不如前者优化。算法中最后两个 **while** 语句, 不可能执行两个, 只能二者取一, 即哪个表尚未到尾, 就将其逆置到结果表中, 即将剩余结点依次前插入到结果表的头结点后面。

2. [题目分析] 循环单链表 L1 和 L2 数据结点个数分别为 m 和 n, 将二者合成一个循环单链表时, 需要将一个循环链表的结点 (从第一元素结点到最后一个结点) 插入到另一循环链表的第一元素结点前即可。题目要求“用最快速度将两表合并”, 因此应找结点个数少的链表查其尾结点。

```
LinkedList Union(LinkedList L1,L2;int m,n)
// L1 和 L2 分别是两循环单链表的头结点的指针, m 和 n 分别是 L1 和 L2 的长度。
// 本算法用最快速度将 L1 和 L2 合并成一个循环单链表。
{if(m<0||n<0) {printf("表长输入错误\n"); exit(0);}
  if(m<n)           // 若 m<n, 则查 L1 循环单链表的最后一个结点。
  {if(m==0)return(L2); // L1 为空表。
    else{p=L1;
      while(p->next!=L1) p=p->next; // 查最后一个元素结点。
      p->next=L2->next; // 将 L1 循环单链表的元素结点插入到 L2 的第一元素结点前。
      L2->next=L1->next;
      free(L1); // 释放无用头结点。
    }
  } // 处理完 m<n 情况
  else // 下面处理 L2 长度小于等于 L1 的情况
  {if(n==0)return(L1); // L2 为空表。
    else{p=L2;
      while(p->next!=L2) p=p->next; // 查最后元素结点。
      p->next=L1->next; // 将 L2 的元素结点插入到 L1 循环单链表的第一元素结点前。
```

```
L1->next=L2->next;
free(L2); // 释放无用头结点。
```

```
}
```

```
// 算法结束。
```

3. [题目分析] 本题基本类似于上面第 7 题，不同之处有二。一是带头结点，二是分解后的两个链表，一个是数据值小于 0，另一个是数据值大于 0。由于没明确要求用类 PASCAL 书写算法，故用 C 书写如下。

```
void DisCreat1(LinkedList A)
```

// A 是带头结点的单链表，链表中结点的数据类型为整型。本算法将 A 分解成两个单链表 B 和 C，B 中结点的数据小于零，C 中结点的数据大于零。

```
{B=A;
C=(LinkedList )malloc(sizeof(LNode)); // 为 C 申请结点空间。
C->next=null // C 初始化为空表。
p=A->next; // p 为工作指针。
B->next=null; // B 表初始化。
while(p!=null)
{r=p->next; // 暂存 p 的后继。
if (p->data<0) // 小于 0 的放入 B 表。
{p->next=B->next; B->next=p;} // 将小于 0 的结点链入 B 表。
else {p->next=C->next; C->next=p;}
p=r; // p 指向新的待处理结点。
}
```

```
// 算法结束。
```

[算法讨论] 因为本题并未要求链表中结点的数据值有序，所以算法中采取最简单方式：将新结点前插到头结点后面（即第一元素之前）。

4. [题目分析] 题目要求重排 n 个元素且以顺序存储结构存储的线性表，使得所有值为负数的元素移到正数元素的前面。这可采用快速排序的思想来实现，只是提出暂存的第一个元素（枢轴）并不作为以后的比较标准，比较的标准是元素是否为负数。

```
int Rearrange (SeqList a; int n)
```

// a 是具有 n 个元素的线性表以顺序存储结构存储，线性表的元素是整数。本算法重排线性表 a，
// 使所有值为负数的元素移到所有值为正数的数的前面。

```
{i=0; j=n-1; // i, j 为工作指针（下标），初始指向线性表 a 的第 1 个和第 n 个元素。
t=a[0]; // 暂存枢轴元素。
while(i<j)
{while(i<j && a[j]>=0) j--; // 若当前元素为大于等于零，则指针前移。
if(i<j){a[i]=a[j]; i++;} // 将负数前移。
while(i<j && a[i]<0) i++; // 当前元素为负数时指针后移。
if(i<j) a[j--]=a[i]; // 正数后移。 }
a[i]=t; // 将原第一元素放到最终位置。
}
```

[算法讨论] 本算法时间复杂度为 $O(n)$ 。算法只是按题目要求把正负数分开，如要求统计负数和大于等于零的个数，则最后以 t 来定。如 t 为负数，则 0 至 i 共 $i+1$ 个负数， $n-1-i$ 个正数（包括零）。另外，题目并未提及零的问题，笔者将零放到正数一边。对此问题的扩充是若元素包含正数、负数和零，并要求按负数、零、正数的顺序重排线性表，统计负数、零、正数的个数。请读者利用上面解题思想自行解答。

5. [题目分析] 本题要求将链表中数据域值最小的结点移到链表的最前面。首先要查找最小值结点。将其移到链表最前面，实质上是将该结点从链表上摘下（不是删除并回收空间），再插入到链表的最前面。

```
LinkedList delinsert (LinkedList list)
```

```

//list 是非空线性链表，链结点结构是 (data, link)，data 是数据域，link 是链域。
// 本算法将链表中数据域值最小的那个结点移到链表的最前面。
p=list->link; // p 是链表的工作指针
pre=list;     // pre 指向链表中数据域最小值结点的前驱。
q=p;         // q 指向数据域最小值结点，初始假定是第一结点
while (p->link!=null)
{
    if (p->link->data<q->data) {pre=p; q=p->link; } // 找到新的最小值结点;
    p=p->link;
}
if (q!=list->link) // 若最小值是第一元素结点，则不需再操作
{
    pre->link=q->link; // 将最小值结点从链表上摘下;
    q->link= list->link; // 将 q 结点插到链表最前面。
    list->link=q;
}
} // 算法结束

```

[算法讨论] 算法中假定 list 带有头结点，否则，插入操作变为 $q \rightarrow \text{link} = \text{list}$; $\text{list} = q$ 。

6. [题目分析] 在递增有序的顺序表中插入一个元素 x ，首先应查找待插入元素的位置。因顺序表元素递增有序，采用折半查找法比顺序查找效率要高。查到插入位置后，从此位置直到线性表尾依次向后移动一个元素位置，之后将元素 x 插入即可。

```

void Insert (ElemType A[], int size, ElemType x)
// A 是有 size 个元素空间目前仅有 num (num<size) 个元素的线性表。本算法将元素 x 插入到线性表中，并保持线性表的有序性。
{
    low=1; high=num; //题目要求下标从 1 开始
    while (low<=high) // 对分查找元素 x 的插入位置。
    {
        mid= (low+high) /2;
        if (A[mid]==x) {low=mid+1; break; }
        else if (A[mid]>x) high=mid-1 ; else low=mid+1 ;
    }
    for (i=num; i>=low; i--) A[i+1]=A[i]; // 元素后移。
    A[i+1]=x; // 将元素 x 插入。
} // 算法结束。

```

[算法讨论] 算法中当查找失败（即线性表中无元素 x ）时，变量 low 在变量 $high$ 的右面 ($low=high+1$)。移动元素从 low 开始，直到 num 为止。特别注意不能写成 $\text{for } (i=low; i \leq num; i++) A[i+1]=A[i]$ ，这是一些学生容易犯的错误。另外，题中未说明若表中已有值为 x 的元素时不再插入，故安排在 $A[mid] = x$ 时，用 $low (=mid+1)$ 记住位置，以便后面统一处理。查找算法时间复杂度为 $O(\log n)$ ，而插入时的移动操作时间复杂度为 $O(n)$ ，若用顺序查找，则查找的时间复杂度亦为 $O(n)$ 。

7. [题目分析] 在无序的单链表上，查找最小值结点，要查遍整个链表，初始假定第一结点是最小值结点。当找到最小值结点后，判断数据域的值是否是奇数，若是，则“与其后继结点的值相交换”即仅仅交换数据域的值，用三个赋值语句即可交换。若与后继结点交换位置，则需交换指针，这时应知道最小值结点的前驱。至于删除后继结点，则通过修改最小值结点的指针域即可。

[算法设计]

```

void MiniValue (LinkedList la)
// la 是数据域为正整数且无序的单链表，本算法查找最小值结点且打印。若最小值结点的数值是奇数，则与后继结点值交换；否则，就删除其直接后继结点。
{
    p=la->next; // 设 la 是头结点的头指针，p 为工作指针。
    pre=p;      // pre 指向最小值结点，初始假定首元结点值最小。
}

```

```

while (p->next!=null) //p->next 是待比较的当前结点。
{
    if (p->next->data<pre->data) pre=p->next;
    p=p->next; // 后移指针
}
printf ("最小值=%d\n", pre->data);
if (pre->data%2!=0) // 处理奇数
    if (pre->next!=null) // 若该结点没有后继, 则不必交换
        {t=pre->data; pre->data=pre->next->data; pre->next->data=t; } // 交换完毕
else // 处理偶数情况
    if (pre->next!=null) // 若最小值结点是最后一个结点, 则无后继
        {u=pre->next; pre->next=u->next; free (u); } // 释放后继结点空间

```

8. [题目分析] 留下三个链表中公共数据, 首先查找两表 A 和 B 中公共数据, 再去 C 中找有无该数据。要消除重复元素, 应记住前驱, 要求时间复杂度 $O(m+n+p)$, 在查找每个链表时, 指针不能回溯。

LinkedList Common (LinkedList A, B, C)

// A, B 和 C 是三个带头结点且结点元素值非递减排列的有序表。本算法使 A 表仅留下三个表均包含的结点, 且结点值不重复, 释放所有结点。

```

{pa=A->next; pb=B->next; pc=C->next; // pa,pb 和 pc 分别是 A,B 和 C 三个表的工作指针。
pre=A; // pre 是 A 表中当前结点的前驱结点的指针。
while (pa && pb && pc) // 当 A,B 和 C 表均不空时, 查找三表共同元素
{
    while (pa && pb)
    {
        if (pa->data<pb->data) {u=pa; pa=pa->next; free (u); } // 结点元素值小时, 后移指针。
        else if (pa->data>pb->data) pb=pb->next;
        else if (pa && pb) // 处理 A 和 B 表元素值相等的结点
        {
            while (pc && pc->data<pa->data) pc=pc->next;
            if (pc)
            {
                if (pc->data>pa->data) // pc 当前结点值与 pa 当前结点值不等, pa 后移指针。
                {
                    u=pa; pa=pa->next; free (u); }
                else // pc, pa 和 pb 对应结点元素值相等。
                {
                    if (pre==A) { pre->next=pa; pre=pa; pa=pa->next; } // 结果表中第一个结点。
                    else if (pre->data==pa->data) // (处理) 重复结点不链入 A 表
                        {u=pa; pa=pa->next; free (u); }
                }
            }
        }
        else {pre->next=pa; pre=pa; pa=pa->next; } // 将新结点链入 A 表。
        pb=pb->next; pc=pc->next; // 链表的工作指针后移。
    } // else pc, pa 和 pb 对应结点元素值相等
    if (pa==null) pre->next=null; // 原 A 表已到尾, 置新 A 表表尾
    else // 处理原 A 表未到尾而 B 或 C 到尾的情况
    {
        pre->next=null; // 置 A 表表尾标记
        while (pa!=null) // 删除原 A 表剩余元素。
        {
            u=pa; pa=pa->next; free (u); }
    }
}

```

[算法讨论] 算法中 A 表、B 表和 C 表均从头到尾 (严格说 B、C 中最多一个到尾) 遍历一遍, 算法时间复杂度符合 $O(m+n+p)$ 。算法主要由 **while** (pa && pb && pc) 控制。三表有一个到尾则结束循环。算法中查到 A 表与 B 表和 C 表的公共元素后, 又分三种情况处理: 一是三表中第一个公共元素值相等的结点; 第二种情况是, 尽管不是第一结点, 但与前驱结点元素值相同, 不能成为结果表中的结点; 第三种情况是新结点与前驱结点元素值不同, 应链入结果表中, 前驱指针也移至当前结点, 以便与以后元素值相同的公共结点进行比较。算法最后要给新 A 表置结尾标记, 同时若原 A 表没到尾, 还应释放剩余结点所占的存储空间。